Lab 2: Fine-tune vs prompt

Does fine-tuning beat your best prompt?

Aarne Talman

5 November 2026

LDA-T318 | LLMs and Generative AI for NLP | University of Helsinki

Course overview

Wk	Dates	Tuesday	Thursday	Teacher
1	27 / 29 Oct	Language models	Pretraining	Aarne
2	**3 / 5 Nov**	**Post-training**	**Lab 2: tune vs prompt**	**Aarne**
3	10 / 12 Nov	Inference	Lab 3: serve	Aarne
4	17 / 19 Nov	Evaluation	Lab 4: evaluate	Jussi
5	24 / 26 Nov	RAG	Lab 5: RAG	Dmitry
6	1 / 3 Dec	Agents	Lab 6: agent	Dmitry
7	8 / 10 Dec	Course recap	Special topics (TBC)	All

Reusing a task and model across labs is recommended, not required.

There are no intermediate submissions; the final report and code archive determine the grade.

The question

One question

Lab 2

For *your* task, does fine-tuning beat your best prompt?

That is the whole lab. Everything today is here to help you answer it **with numbers you produced yourself**.

Tuesday's lecture gave you the machinery: assistant-only loss, LoRA, catastrophic forgetting, and the decision slide (*behaviour → fine-tune, knowledge → retrieve*). Today you test it on your own task.

What week 2 adds to your project

A prompting baseline and a fine-tuned model, **both scored on the same held-out set**.

You may serve the adapter you train today in week 3, or use another compatible model. Week 4 examines evaluation in more depth.

Start with `lab2-example/README.md`; it contains the baseline, LoRA, evaluation and smoke-test commands.

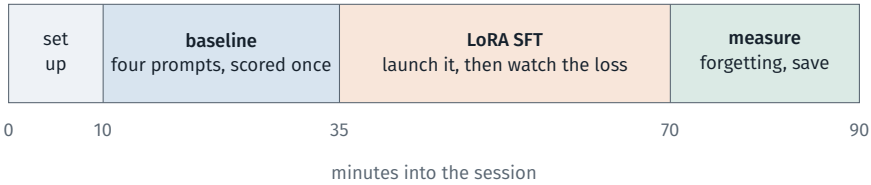
Material for the final report

Setting	Held-out score	What it is
Zero-shot		best of the base or instruct model
Best prompt		choose on dev; report held-out score
Fine-tuned (LoRA SFT)		same held-out set, same decoding
Forgetting check	before, after	50 general-knowledge questions

Plus one loss curve, with training *and* evaluation loss on the same axes.

Plus one paragraph: was it worth it? GPU-hours and human hours spent, against points gained. Would you ship it?

Today, minute by minute



This briefing stops at minute 10. If you are not running something by minute 15, put your hand up.

The training run is the long pole, so **launch it before you finish polishing anything else.** It trains while you write the evaluation code.

PROTOCOL

Run four baselines

	Zero-shot	Few-shot (5 examples)
Base model		
Instruct model		

Compare all four on the **development set**, with the **same decoding parameters** and the **same answer-extraction function**.

Whichever scores highest on dev is *the* baseline. Report its test score only after you have made that choice. Do not report the instruct zero-shot number just because it was the easiest one to run.

A weak baseline fakes a win

Beating a bad prompt tells you nothing. If you have not tried few-shot on the instruct model, you do not have a baseline yet.

Decoding settings and the held-out set

Fix these once, use them everywhere

temperature	0 (greedy)
`max_new_tokens`	long enough for your longest gold answer
seed	fixed and recorded
prompt template	identical across all four
answer extraction	one shared function

Greedy decoding is the right choice here because the task has one correct answer and you want the four runs to be comparable. (Week 3: temperature 0 is *not* deterministic once requests are batched. Today it is close enough.)

Three splits

- **train**: the SFT data, 1–5k examples
- **dev**: where you choose prompts, epochs, learning rate
- **held-out test**: touched once per reported number

Take your few-shot examples from **train or dev**, never from test. Report the split sizes and the metric in the table caption. “Accuracy” on an unnamed subset is not a number anyone can use.

Three ways to silently ruin a baseline

1. Answer extraction. The model answered “She sold 72 clips.” Your parser wanted a bare integer. Score: 0. *Fix:* one extraction function, shared by every run, and ten printed generations next to their extracted answers.

2. A chat template on a base model. The base model never saw `<|im_start|>` or any other control token. Those arrive in post-training. *Fix:* plain completion-style prompts for the base model, `apply_chat_template` only for the instruct model.

3. Contamination. GSM8K, MNLI and MMLU are in everyone’s pretraining mix, so part of your “zero-shot” score may be recall. *Fix:* you cannot decontaminate someone else’s pretraining, so say so in the report. Week 4 treats this properly.

Check the parser before you believe the number

A score of exactly 0.0 is a reason to inspect the extraction code before concluding that the model failed. Before you believe *any* baseline number, print ten raw generations beside their extracted answers and read them.

Parser and template errors often make the baseline too low. Contamination can make it too high. Either error invalidates the comparison.

Doing it

Your data: one conversation per example

```
# one training example = one conversation, in TRL's "messages" format
{"messages": [
  {"role": "system",    "content": "Solve the problem. End with 'Answer: <n>'."},
  {"role": "user",      "content": "Natalia sold clips to 48 friends ..."},
  {"role": "assistant", "content": "48 + 24 = 72. Answer: 72"}]}

from datasets import load_dataset
train = load_dataset("json", data_files="data/train.jsonl")["train"]
dev   = load_dataset("json", data_files="data/dev.jsonl")["train"]
```

- Let TRL apply **the model's own chat template**; templates differ between model families.
- Compute **loss on assistant tokens only**, so the reply is the training target while the prompt remains context.
- Start with **1–5k examples**. Make each assistant turn match the output format your scorer expects.
- Before training, print one rendered example and inspect the exact string and loss mask.

Starting hyperparameters, and the three you may tune

Knob	Start at
LoRA rank	16
LoRA alpha	32 (twice the rank)
LoRA dropout	0.05
target modules	`all-linear`
learning rate	0.0002
schedule	cosine, 3% warmup
epochs	2
effective batch	16 sequences (8×2)
max length	1024 tokens
precision	bf16

You may tune exactly three

1. **learning rate**
2. **number of epochs**
3. **LoRA rank**, the capacity of the adapter (move alpha with it, keeping alpha at twice the rank)

Leave everything else alone. Change *one* knob at a time and log every run in one small table: knob, value, train loss, eval loss, held-out score.

That table is more useful than a better score without a record of what changed.

Source: Common LoRA practice and TRL/PEFT defaults. These are **starting points**, so do not quote any of them in the report as a finding.

When the loss falls and nothing else moves

Lower training loss is not a result

Training loss down, evaluation loss flat or rising, held-out score unchanged: you memorised the training set. With LoRA this happens *fast*,

because you have more trainable parameters than training tokens.

Do the arithmetic for your own run.
`model.print_trainable_parameters()` gives you the numerator. Your token count is roughly (examples) $\times$ (tokens per example).

A plausible shape for a 2B model with rank 16 on all linear layers:

trainable parameters	$\sim$10–40 M
2k examples $\times$ 300 tok	$\sim$0.6 M tok

What to do about it

- plot **train and eval loss together**, always
- stop at the epoch where eval loss turns up
- then reduce epochs, then the learning rate, then *r*, in that order

Source: Parameter counts here are an order-of-magnitude illustration. Print the real number for your own model and report that.

The forgetting check

50 general-knowledge questions, unrelated to your task. An MMLU slice is fine.

Score them **twice**: on the model you started from, and on the fine-tuned model. Same script, same decoding, same extraction.

Treat this as a **regression test**. Nobody cares whether your 2B model knows 42% or 55% of MMLU; what you are reporting is the **difference** between before and after.

A few points down is normal and worth one sentence. Twenty points down means the adapter ate the model: fewer epochs, lower learning rate, lower rank.

Why this is in the lab at all

Tuesday's claim: *always measure what fine-tuning broke, not only what it improved.* GLM-5 ends its pipeline with self-distillation precisely to recover capabilities weakened by the earlier RL stages. Catastrophic forgetting turns up as a line item in production recipes.

50 questions is a blunt instrument

With only 50 right/wrong items, one standard error is about **7 percentage points** when accuracy is near 50%. Report the pair of numbers and do not over-read a 3-point change.

Source: MMLU-as-regression-test framing: Raschka, *Understanding the 4 Main Approaches to LLM Evaluation.* Standard error from the binomial, $n = 50$.

Bonus

Before you leave

The report table and files to keep

Setting	Score	Config
Zero-shot, instruct		greedy
Few-shot 5, instruct		greedy
Zero-shot, base		greedy
Few-shot 5, base		greedy
LoRA SFT		rank 16, 2 epochs
Forgetting, before		50 Q
Forgetting, after		50 Q

Caption it with: dataset, split sizes, metric, model IDs.

Keep for the final submission

- the notebook, run top to bottom
- `sft_lora.py` and the Slurm script
- `results/lab2.json` with every number above
- the loss curve (train *and* eval)
- the LoRA adapter, or its path on Roihu if it is large

There is no intermediate submission. Keep the adapter if you want to serve it in week 3, and retain the results for the final report.

Roihu reminders

You lost two days of compute this week

Roihu was down for scheduled maintenance on **3–4 November**, so there was no homework compute between Tuesday's lecture and now. That is why the baseline block is at the front of today's session rather than assumed as homework.

The lab reservation covers roughly **16:00–20:00**, so runs started in the session can finish after it. Next maintenance: 1–2 December.

Etiquette, unchanged all term

- `gputest` (15 min, short queue) before `gpumedium`
- **close idle GPU Jupyter sessions.** One forgotten session holds a whole GH200 for twelve hours, and we share one allocation
- never train on a login node; it has no GPU and its CPUs are shared
- `seff <jobid>` after every batch job, to see what you actually used

Environment: build and test the course environment on Roihu's ARM login nodes before requesting a GPU. The training configuration uses bf16, which the GH200 supports natively.

Source: CSC Roihu documentation: system description, partitions, course maintenance windows.

Leave this on the wall



- [ ] four baselines run, same decoding
- [ ] best on dev named as *the* baseline
- [ ] ten raw generations read with your own eyes
- [ ] forgetting check scored **before** training
- [ ] LoRA SFT run, train *and* eval loss plotted
- [ ] fine-tuned model scored on the held-out set
- [ ] forgetting check scored after
- [ ] scripts, settings and results saved

Stuck for more than ten minutes? Put your hand up. Appendix slide has the common errors.

Appendix: things that will go wrong

`CUDA out of memory`
Halve `per_device_train_batch_size`, double `gradient_accumulation_steps`, cut `max_length`. Then **restart the kernel**, because a dead run keeps its memory.

Score is exactly 0.00 or 1.00
Answer extraction. Print ten raw generations.

Generations are empty or truncated
`max_new_tokens` too small, or you forgot `add_generation_prompt=True`.

"Tokenizer has no chat template"
You loaded the *base* model. Either prompt it completion-style, or use the instruct tokenizer, and say which in the report.

`pad_token` is `None`
`tok.pad_token = tok.eos_token`, then re-create the trainer.

Loss is `nan`
Learning rate too high, or something is running in fp16. We are bf16 everywhere.

Job stuck in `PD` (pending) forever
Wrong account or partition, or more GPUs than the reservation holds. `squeue --me --start` for an estimate, `scancel` to clear. An empty log usually means `logs/` does not exist.

Training loss suspiciously low from step 1
`assistant_only_loss` is off. You are training on your own prompts.